29. Design a Recommendation Engine

Key Terms:

Term	Full Name	What it does
Collaborative Filtering	User-based recommendations	"Users who liked X also liked Y"
Content-Based Filtering	Feature-based recommendations	Match item features to user preferences
Matrix Factorization	ML technique	Discovers latent factors from sparse user-item matrix
Cold Start	New user/item problem	No history to base recommendations on
ANN	Approximate Nearest Neighbors	Fast similarity search in embedding space
Two-Tower Model	Deep learning architecture	Separate user and item embedding networks
SVD	Singular Value Decomposition	Matrix factorization algorithm
ALS	Alternating Least Squares	Scalable matrix factorization for Spark

Problem Statement

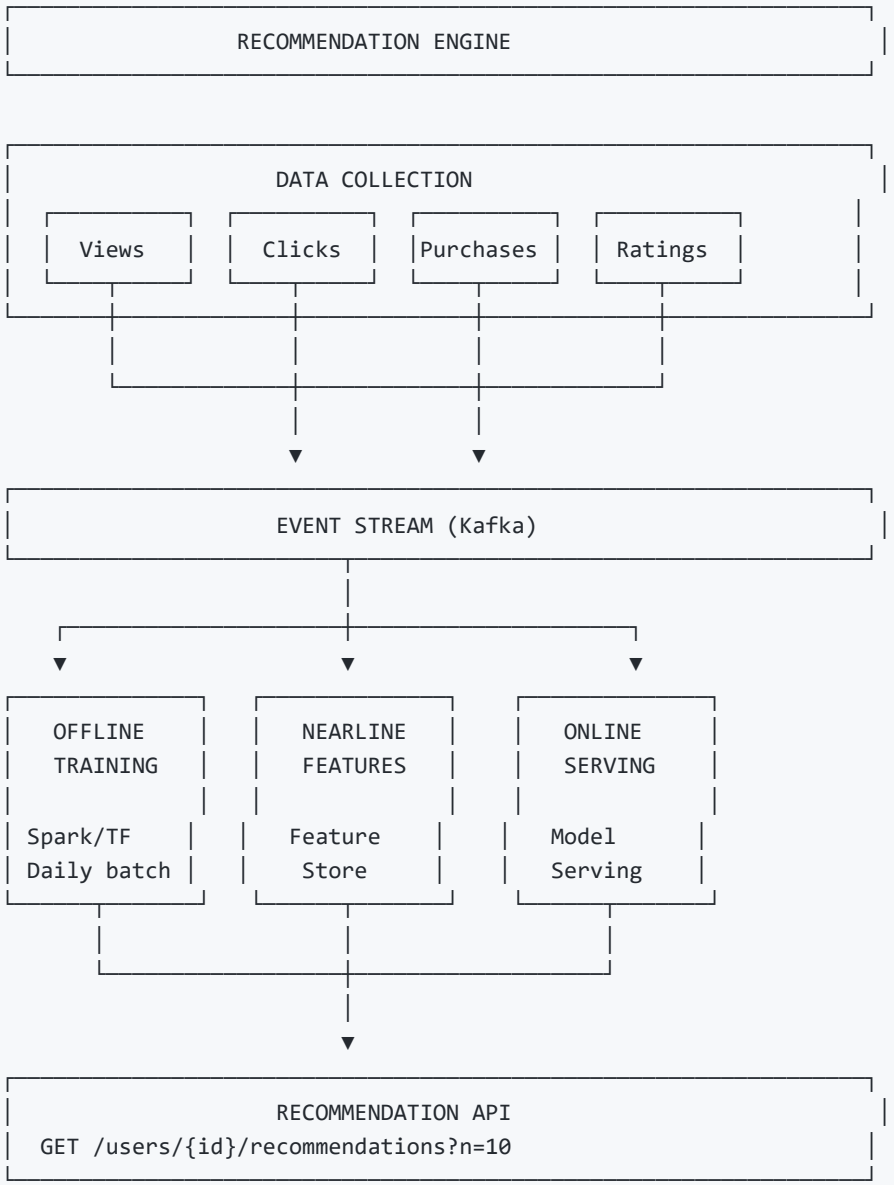
Design a system that provides personalized recommendations for products, content, or connections based on user behavior and preferences.

Clarifying Questions to Ask

1. **Domain**: Products, movies, music, friends, or content?
2. **Scale**: How many users? Items? (1M users, 10M items?)
3. **Latency**: Real-time (<100ms) or precomputed?
4. **Cold start**: How to handle new users/items?
5. **Freshness**: How often should recommendations update?

6. **Explainability**: Need to explain why recommended?
7. **Diversity**: Balance relevance vs. discovery?

High-Level Design



API Design

Recommendation APIs

Method	Endpoint	Description
`GET`	`/api/v1/recommendations?userId={id}&type={type}&limit=20`	Get personalized recommendations

`GET`	`/api/v1/recommendations/similar/{itemId}`	Get similar items
`GET`	`/api/v1/recommendations/trending?category={cat}`	Get trending items

User Interaction APIs (for training)

Method	Endpoint	Description
`POST`	`/api/v1/events`	Log user interaction event
`POST`	`/api/v1/events/batch`	Log batch of interaction events

Event Payload

```
POST /api/v1/events
{
  "user_id": "user_123",
  "item_id": "item_456",
  "event_type": "click",      // click, view, purchase, rate
  "rating": 4.5,              // optional
  "context": {
    "source": "homepage",
    "position": 3,
    "session_id": "sess_789"
  },
  "timestamp": 1672531200000
}
```

Model Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/models/retrain`	Trigger model retraining
`GET`	`/api/v1/models/active`	Get active model info
`GET`	`/api/v1/models/{modelId}/metrics`	Get model accuracy metrics

Recommendation Approaches

```
1. COLLABORATIVE FILTERING (Users who liked X also liked Y)
┌─────────────────────────────────────────────────────────────┐
│  USER-BASED:                                                │
│    Find users similar to target user                        │
```

```
│    Recommend items those similar users liked                    │
│                                                                  │
│    User A likes: [1, 2, 3]                                       │
│    User B likes: [1, 2, 4]     ← Similar to A                    │
│    User C likes: [5, 6, 7]     ← Different from A                │
│                                                                  │
│    Recommend to A: item 4 (from similar user B)                  │
│                                                                  │
│  ITEM-BASED:                                                     │
│    Find items similar to what user liked                         │
│    More stable (items don't change as fast as user behavior)     │
│                                                                  │
│    User likes item 1                                             │
│    Item 1 similar to: [2, 3, 5]                                  │
│    Recommend: [2, 3, 5]                                          │
└──────────────────────────────────────────────────────────────────┘
```

2. CONTENT-BASED FILTERING (Items similar in features)

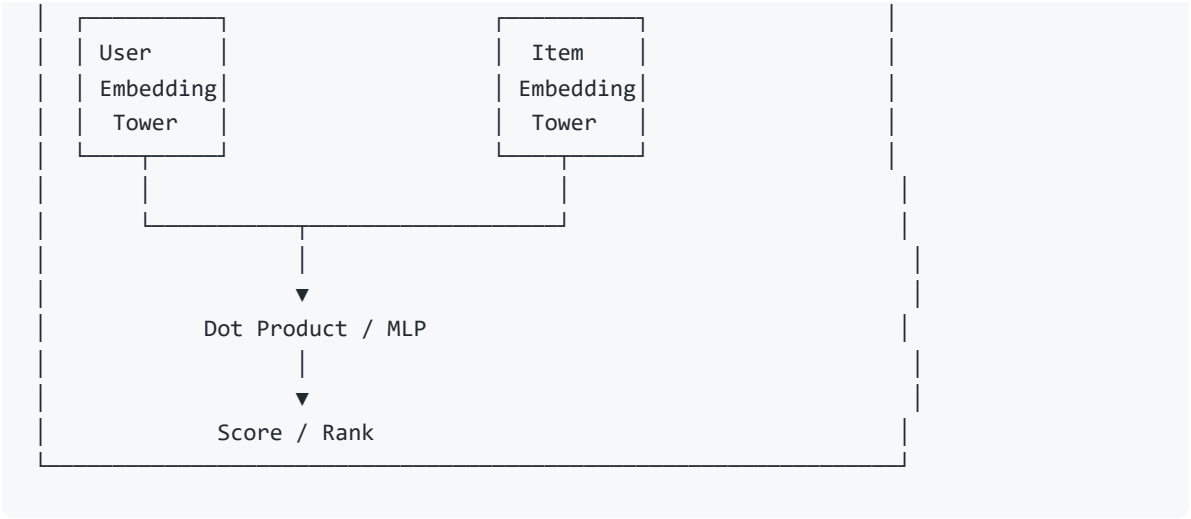
```
┌──────────────────────────────────────────────────────────────────┐
│  Extract item features:                                          │
│    Movie: {genre: "action", actors: ["Tom"], director: "James"}  │
│                                                                  │
│  User profile: Aggregate features of liked items                 │
│    User likes action movies with Tom                             │
│                                                                  │
│  Recommend: Items matching user profile                          │
│    Other action movies with Tom                                  │
│                                                                  │
│  Pros: Works for new items (no cold start for items)             │
│  Cons: Limited discovery, needs good features                    │
└──────────────────────────────────────────────────────────────────┘
```

3. MATRIX FACTORIZATION (Discover latent factors)

```
┌──────────────────────────────────────────────────────────────────┐
│  User-Item matrix (sparse):                                      │
│         Item1  Item2  Item3  Item4                               │
│  User1    5      ?      3      ?                                 │
│  User2    ?      4      ?      2                                 │
│  User3    3      ?      ?      5                                 │
│                                                                  │
│  Factorize into: Users × Latent × Items                          │
│  Predict missing ratings: User1 × Item2                          │
│                                                                  │
│  Algorithms: SVD, ALS, NMF                                       │
└──────────────────────────────────────────────────────────────────┘
```

4. DEEP LEARNING (Modern approach)

```
┌──────────────────────────────────────────────────────────────────┐
│  Two-Tower Model:                                                │
│                                                                  │
│   ┌──────────┐                    ┌──────────┐                   │
│   │ User     │                    │  Item    │                   │
│   │ Features │                    │ Features │                   │
│   └────┬─────┘                    └────┬─────┘                   │
│        │                               │                         │
│        ▼                               ▼                         │
```



Cold Start Solutions

```
NEW USER (No history):
┌──────────────────────────────────────────────────────────────┐
│  1. Popular items: Recommend trending/popular items          │
│  2. Onboarding: Ask preferences during signup                │
│  3. Demographics: Use age, location as initial signal        │
│  4. Content-based: If user searches "sci-fi", use that       │
└──────────────────────────────────────────────────────────────┘

NEW ITEM (No ratings):
┌──────────────────────────────────────────────────────────────┐
│  1. Content features: Use item metadata to find similar items  │
│  2. Exploration: Show to some users to gather signals        │
│  3. Expert curation: Manual categorization/tagging           │
└──────────────────────────────────────────────────────────────┘
```

Real-Time vs. Batch

```
BATCH PIPELINE (Daily/Hourly):
┌──────────────────────────────────────────────────────────────┐
│  1. Train model on historical data                           │
│  2. Generate recommendations for all users                   │
│  3. Store in Redis/Database                                  │
│  4. API serves precomputed recommendations                   │
│                                                              │
│  Pros: Simple, handles large-scale computation               │
│  Cons: Not real-time, stale recommendations                  │
└──────────────────────────────────────────────────────────────┘

REAL-TIME (Online):
┌──────────────────────────────────────────────────────────────┐
│  1. User opens app                                           │
│  2. Fetch user features from feature store                   │
│  3. Candidate generation: ANN search for similar items       │
```

```
│  4. Ranking: ML model scores candidates                          │
│  5. Return top-K                                                 │
│                                                                    │
│  Pros: Fresh recommendations, personalized                       │
│  Cons: Complex, latency challenges                               │
└──────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Approach	Collaborative	Content	**Hybrid** - best of both
Serving	Batch	Real-time	**Batch + real-time rerank**
Diversity	Pure relevance	Exploration	**Balance** - avoid filter bubble
Cold start	Popular	Onboarding	**Both** - graceful degradation

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Event streaming for user behavior data	Tech Guide
Redis	Feature store and recommendation cache	Tech Guide
Flink/Spark	Offline model training and batch processing	Tech Guide

---

How to Explain It Well

> **Opening**: "Recommendations combine collaborative filtering (user behavior) and content-based (item features). I'll design a two-tower model with real-time serving."

> **Key talking points**:
>
> - "Collaborative filtering: users who liked X also liked Y"
> - "Matrix factorization discovers latent factors without explicit features"
> - "Two-tower model: precompute item embeddings, compute user online"
> - "Cold start: popular items for new users, content features for new items"
> - "Balance relevance and diversity to avoid filter bubbles"